

Inter-Task Communication (ITC) in FreeRTOS

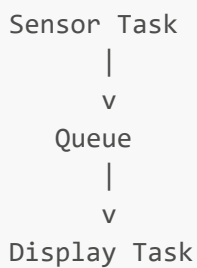
Inter-Task Communication (ITC) is the mechanism used for exchanging data and synchronization information between multiple tasks in a FreeRTOS application.

Since tasks run independently and may share resources, ITC helps them communicate safely and efficiently.

Why Inter-Task Communication is Required?

- Exchange data between tasks.
- Synchronize task execution.
- Notify events between tasks.
- Protect shared resources.
- Avoid race conditions.

Example:



FreeRTOS Inter-Task Communication Mechanisms

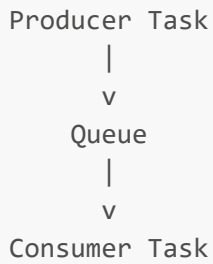
1. Queues

- Used to transfer data between tasks.
- FIFO (First In First Out) mechanism.
- Can transfer structures, integers, pointers, etc.
- Most commonly used ITC mechanism.

APIs

```
xQueueCreate()  
xQueueSend()  
xQueueReceive()
```

Example



Applications:

- Sensor Data Transfer
- UART Communication
- Command Processing

2. Semaphores

- Used for synchronization.
- Does not transfer data.
- Used to signal events.

Types

Binary Semaphore

0 = Not Available
1 = Available

Used for:

- ISR to Task synchronization
- Event signaling

Counting Semaphore

- Maintains a count value.
- Used when multiple events can occur.

APIs

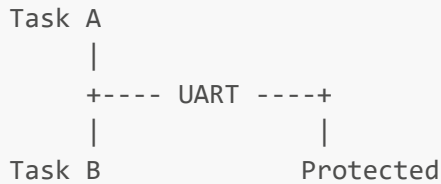
```

xSemaphoreCreateBinary()
xSemaphoreTake()
xSemaphoreGive()
  
```

3. Mutex

- Special type of binary semaphore.
- Used for resource protection.
- Prevents multiple tasks from accessing the same resource simultaneously.

Example:



APIs

```

xSemaphoreCreateMutex()
xSemaphoreTake()
xSemaphoreGive()
  
```

Applications:

- UART Protection
- LCD Access
- Shared Memory Protection

4. Task Notifications

- Lightweight alternative to semaphores.
- Fastest communication mechanism in FreeRTOS.
- Each task has its own notification value.

APIs

```

xTaskNotify()
xTaskNotifyGive()
ulTaskNotifyTake()
  
```

Applications:

- ISR to Task Communication
- Event Notification

5. Event Groups

- Used to manage multiple events using bit flags.

- One task can wait for one or more events.

Example:

```
Bit0 = Sensor Ready
Bit1 = UART Ready
Bit2 = Network Ready
```

APIs

```
xEventGroupCreate()
xEventGroupSetBits()
xEventGroupWaitBits()
```

Applications:

- System Initialization
- Multiple Event Synchronization

Comparison of ITC Mechanisms

Mechanism	Data Transfer	Synchronization	Resource Protection
Queue	Yes	Yes	No
Semaphore	No	Yes	No
Mutex	No	Yes	Yes
Task Notification	Limited	Yes	No
Event Group	No	Yes	No

Typical Usage

Queue

```
Task A ---- Data ----> Task B
```

Semaphore

```
ISR ---- Event ----> Task
```

Mutex

```

Task A
  \
   > Shared UART
  /
Task B

```

Task Notification

```
ISR ---- Notification ----> Task
```

Event Group

```

Event1
Event2 ---> Event Group ---> Task
Event3

```

Interview Questions

What is Inter-Task Communication?

Inter-Task Communication is the mechanism used by tasks to exchange data and synchronize execution.

Which ITC mechanism is most commonly used?

Queue.

Which ITC mechanism is fastest?

Task Notification.

Difference between Semaphore and Mutex?

- Semaphore is used for synchronization.
- Mutex is used for protecting shared resources and supports priority inheritance.

Which mechanism is preferred for ISR-to-Task communication?

- Binary Semaphore
- Task Notification (preferred due to lower overhead)

Queue in FreeRTOS

A Queue is one of the most commonly used Inter-Task Communication (ITC) mechanisms in FreeRTOS.

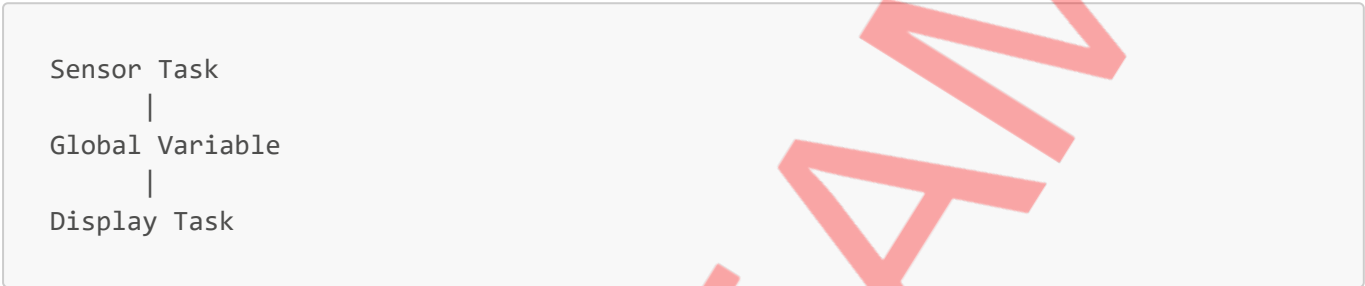
A queue allows tasks, ISRs, and software timers to safely exchange data.

Queues operate on FIFO (First In First Out) principle.

Why Queue is Required?

Suppose a sensor task reads temperature data and a display task shows the value.

Without Queue:



Problems:

- Race conditions
- Data corruption
- Synchronization issues

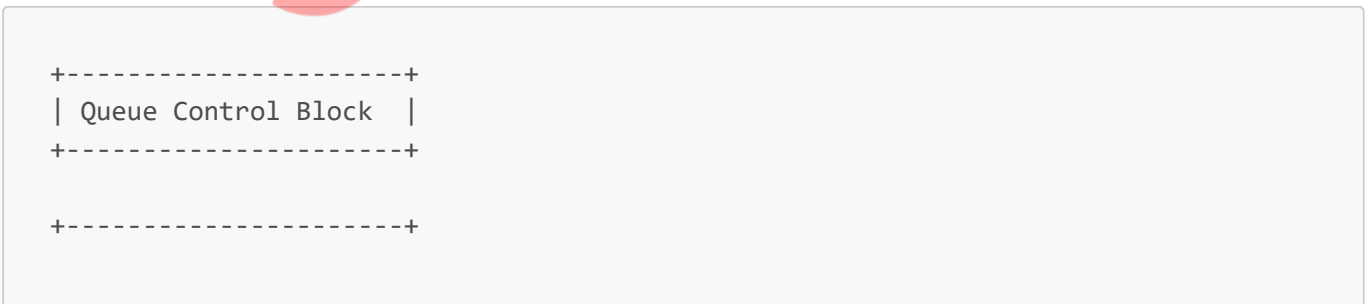
With Queue:

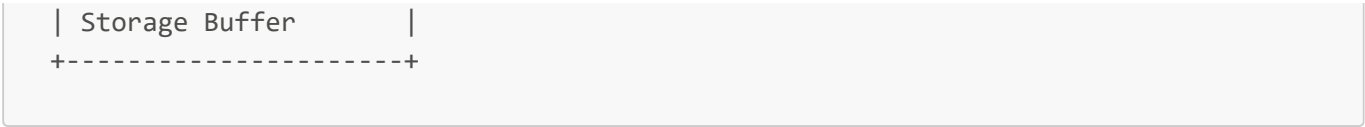


Queue provides safe communication and synchronization.

Queue Internal Structure

A Queue consists of:





Queue Control Block stores:

- Queue Length
- Item Size
- Read Index
- Write Index
- Waiting Tasks List

Storage Buffer stores actual data.

Queue Creation

API

```
QueueHandle_t xQueueCreate(  
    UBaseType_t uxQueueLength,  
    UBaseType_t uxItemSize);
```

Parameters

Parameter	Description
uxQueueLength	Maximum number of items queue can hold
uxItemSize	Size of each item in bytes

Return Value

Value	Description
NULL	Queue creation failed
Valid QueueHandle_t	Queue created successfully

Sending Data to Queue

API

```

 BaseType_t xQueueSend(
     QueueHandle_t xQueue,
     const void *pvItemToQueue,
     TickType_t xTicksToWait);

```

Parameters

Parameter	Meaning
xQueue	Queue handle
pvItemToQueue	Address of data to send
xTicksToWait	Time to wait if queue is full

Return Values

Value	Description
pdPASS	Successfully inserted
errQUEUE_FULL	Queue full

Receiving Data

API

```

 BaseType_t xQueueReceive(
     QueueHandle_t xQueue,
     void *pvBuffer,
     TickType_t xTicksToWait);

```

Parameters

Parameter	Meaning
xQueue	Queue handle
pvBuffer	Buffer to store received item
xTicksToWait	Wait time if queue empty

Queue Peek

Peek reads data without removing it.

API

```
xQueuePeek()
```

Queue Delete

API

```
void vQueueDelete(  
    QueueHandle_t xQueue);
```

Used to release queue resources.

Queue From ISR

Normal Queue APIs should NOT be used inside ISR.

Use ISR-safe APIs.

Send From ISR

```
BaseType_t xQueueSendFromISR(  
    QueueHandle_t xQueue,  
    const void *pvItemToQueue,  
    BaseType_t *pxHigherPriorityTaskWoken);
```

Parameters

Parameter	Meaning
xQueue	Queue Handle
pvItemToQueue	Data to send
pxHigherPriorityTaskWoken	Indicates if higher-priority task became ready

Example

```
BaseType_t xHigherPriorityTaskWoken = pdFALSE;
```

```
xQueueSendFromISR(  
    xQueue,  
    &data,  
    &xHigherPriorityTaskWoken);  
  
portYIELD_FROM_ISR(  
    xHigherPriorityTaskWoken);
```

Queue Full Condition

Example:

```
Length = 3  
  
[A][B][C]
```

Another Send:

```
Queue Full
```

Behavior depends on:

```
xTicksToWait
```

Queue Empty Condition

Example:

```
[ ]
```

Receive operation:

```
Task Blocks
```

until data arrives or timeout occurs.

Queue Usage with Structures

```
typedef struct
{
    int temp;
    int humidity;
}SensorData_t;
```

Queue Creation:

```
QueueHandle_t xSensorQueue;

xSensorQueue =
    xQueueCreate(
        5,
        sizeof(SensorData_t));
```

Send:

```
SensorData_t data;

xQueueSend(
    xSensorQueue,
    &data,
    portMAX_DELAY);
```

Receive:

```
SensorData_t rxData;

xQueueReceive(
    xSensorQueue,
    &rxData,
    portMAX_DELAY);
```

Typical Applications

Sensor Task -> Display Task

```
Sensor Task
|
v
Queue
|
```

v
Display Task

UART RX -> Processing Task

UART ISR
|
v
Queue
|
v
Parser Task

Producer Consumer Model

Producer
|
v
Queue
|
v
Consumer

Advantages

- Safe data transfer.
- Synchronization.
- Multiple producers/consumers.
- ISR support.
- Blocking operations.

Limitations

- Memory overhead.
- Copy operation overhead.
- Not suitable for very large data blocks.

Interview Questions

What is a Queue?

Queue is a FIFO-based inter-task communication mechanism used for safe data exchange between tasks.

Difference between Queue and Semaphore?

Queue transfers data.

Semaphore transfers synchronization signals.

Why use Queue instead of Global Variables?

Queues provide synchronization and prevent race conditions.

Which Queue API should be used inside ISR?

```
xQueueSendFromISR()
```

and

```
xQueueReceiveFromISR()
```

What happens if Queue is Full?

Send operation blocks or fails depending on `xTicksToWait`.

What happens if Queue is Empty?

Receive operation blocks or times out.

Semaphore in FreeRTOS

A Semaphore is a synchronization mechanism used to signal events between tasks or between an ISR and a task.

A semaphore does NOT carry data.

Think of a semaphore as a token.

```
Token Available  
|
```

V
Task Can Proceed

Why Semaphore?

Without Semaphore:

Task Polls Flag

Problems:

- CPU waste
- Increased latency

With Semaphore:

Task Blocks
|
Semaphore Given
|
Task Wakes Up

Types of Semaphores

Binary Semaphore

Binary semaphore can have only two states:

0 -> Not Available
1 -> Available

Used for:

- Event notification
- ISR-to-task communication

API: Binary Semaphore Creation

```
SemaphoreHandle_t xSemaphoreCreateBinary(void);
```

Return Value

Value	Meaning
NULL	Creation Failed
Handle	Success

Taking a Semaphore

API

```
BaseType_t xSemaphoreTake(
    SemaphoreHandle_t xSemaphore,
    TickType_t xTicksToWait);
```

Parameters

Parameter	Meaning
xSemaphore	Semaphore handle
xTicksToWait	Time to wait

Giving a Semaphore

API

```
BaseType_t xSemaphoreGive(
    SemaphoreHandle_t xSemaphore);
```

Example

```
xSemaphoreGive(
    xBinarySem);
```

ISR Example

ISR:

```
void EXTI0_IRQHandler(void)
{
```

```

        BaseType_t xHigherPriorityTaskWoken =
                        pdFALSE;

        xSemaphoreGiveFromISR(
            xBinarySem,
            &xHigherPriorityTaskWoken);

        portYIELD_FROM_ISR(
            xHigherPriorityTaskWoken);
    }

```

Task:

```

while(1)
{
    xSemaphoreTake(
        xBinarySem,
        portMAX_DELAY);

    ProcessButton();
}

```

Counting Semaphore

Used when multiple events can occur.

Example:

Printer Resource Count = 3

Count:

3 -> 2 -> 1 -> 0

API

```

SemaphoreHandle_t
xSemaphoreCreateCounting(
    UBaseType_t uxMaxCount,
    UBaseType_t uxInitialCount);

```


Parameters

Parameter	Meaning
uxMaxCount	Maximum count
uxInitialCount	Starting count

Typical Applications

- Button Interrupt
- ADC Complete
- DMA Complete
- UART Reception Event
- Event Counting

Important Interview Questions

Does Semaphore Transfer Data?

No.

Semaphore transfers only synchronization information.

Why Binary Semaphore?

For event signaling.

Why Counting Semaphore?

For counting multiple occurrences of an event.

Which API is used from ISR?

`xSemaphoreGiveFromISR()`

Mutex

What is a Mutex?

Mutex stands for Mutual Exclusion.

A mutex is a synchronization mechanism used to protect a shared resource from simultaneous access by multiple tasks.

Only one task can own a mutex at a time.

Why Mutex is Required?

Without Mutex:

```
Task A : HELLO  
Task B : WORLD
```

Possible Output:

```
HEWLORLD
```

Data gets corrupted because both tasks access UART simultaneously.

With Mutex:

```
Task A  
Take Mutex  
Print HELLO  
Give Mutex  
  
Task B  
Take Mutex  
Print WORLD  
Give Mutex
```

Output:

```
HELLOWORLD
```

Mutex Creation

API

```
SemaphoreHandle_t xSemaphoreCreateMutex(void);
```

Parameters

None

Return Value

Return Value	Meaning
NULL	Creation Failed
Handle	Success

Taking a Mutex

API

```
BaseType_t xSemaphoreTake(
    SemaphoreHandle_t xMutex,
    TickType_t xTicksToWait);
```

Parameters

Parameter	Meaning
xMutex	Mutex Handle
xTicksToWait	Wait Time

Giving a Mutex

API

```
BaseType_t xSemaphoreGive(
    SemaphoreHandle_t xMutex);
```

Parameters

Parameter	Meaning
xMutex	Mutex Handle

Priority Inversion

Example:

```
Task H Priority = 3
Task M Priority = 2
Task L Priority = 1
```

Task L owns mutex.

Task H waits for mutex.

Task M keeps running.

This condition is called Priority Inversion.

Priority Inheritance

FreeRTOS temporarily increases the priority of the mutex owner until the mutex is released.

Advantages

- Prevents race conditions.
 - Supports priority inheritance.
 - Protects shared resources.
-

Limitations

- Cannot transfer data.
 - Cannot be used in ISR.
-

Interview Questions

What is a Mutex?

A synchronization object used to protect shared resources.

Can Mutex be used in ISR?

No.

What problem does Priority Inheritance solve?

Priority Inversion.

Task Notifications

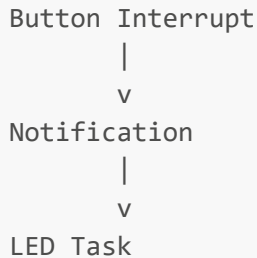
Task Notifications are a lightweight mechanism used for communication and synchronization between tasks or between an ISR and a task.

Unlike semaphores or queues, no separate kernel object is created. Each task already contains a notification value inside its TCB (Task Control Block).

Why Task Notifications?

- Faster than queues and semaphores.
- Lower memory consumption.
- Ideal for ISR-to-task communication.

Example:



#3 Common APIs

Send Notification

```

BaseType_t xTaskNotifyGive(
    TaskHandle_t xTaskToNotify);
  
```

Parameter

Parameter	Meaning
xTaskToNotify	Task handle to notify

Wait for Notification

```

uint32_t ulTaskNotifyTake(
    BaseType_t xClearCountOnExit,
    TickType_t xTicksToWait);
  
```

Parameters

Parameter	Meaning
xClearCountOnExit	Clear notification count after receiving
xTicksToWait	Time to wait

ISR Example

ISR:

```
void EXTI0_IRQHandler(void)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    vTaskNotifyGiveFromISR(
        xTaskHandle,
        &xHigherPriorityTaskWoken);

    portYIELD_FROM_ISR(
        xHigherPriorityTaskWoken);
}
```

Task:

```
while(1)
{
    ulTaskNotifyTake(
        pdTRUE,
        portMAX_DELAY);

    ProcessEvent();
}
```

Task Notification vs Semaphore

Feature	Notification	Semaphore
Speed	Fastest	Slower
RAM Usage	Very Low	Higher
Object Creation	Not Required	Required
ISR Support	Yes	Yes

Advantages

- No separate kernel object required.
- Very fast execution.
- Minimal memory overhead.
- Excellent for ISR-to-task signaling.
- Lower CPU overhead compared to semaphores.

Limitations

- Primarily designed for one-to-one communication.
- Cannot buffer multiple data items like a queue.
- Less flexible than queues for data transfer.

Typical Applications

Button Interrupt

```
Button ISR
  |
  v
Notification
  |
  v
LED Task
```

DMA Transfer Complete

```
DMA ISR
  |
  v
Notification
  |
  v
Processing Task
```

ADC Conversion Complete

```
ADC ISR
  |
  v
Notification
  |
  v
Sensor Task
```

Interview Questions

What is Task Notification?

A lightweight synchronization mechanism built into every task's TCB.

Why is Task Notification faster than a Semaphore?

Because it does not require a separate kernel object and operates directly on the task's notification value.

What is the fastest Inter-Task Communication mechanism in FreeRTOS?

Task Notification.

Which API is commonly used for ISR-to-task notification?

```
vTaskNotifyGiveFromISR()
```

Which API is commonly used by the task to wait for a notification?

```
ulTaskNotifyTake()
```

Memory Trick

```
Queue      -> Transfer Data
Semaphore  -> Signal Event
Mutex      -> Protect Resource
Notification -> Fast Event Signal
```

Deferred Processing

What is Deferred Processing?

Deferred Processing is a technique in which time-consuming work is moved out of the Interrupt Service Routine (ISR) and executed later by a task.

The ISR performs only critical operations and notifies a task to complete the remaining processing.

Why Deferred Processing is Required?

ISRs should execute as quickly as possible.

Long ISR execution causes:

- Increased interrupt latency
- Missed interrupts
- Poor responsiveness
- Missed real-time deadlines

Bad Design:

```

Interrupt
|
Long ISR
|
Heavy Processing
|
ISR Exit

```

Good Design:

```

Interrupt
|
Short ISR
|
Signal Task
|
ISR Exit
|
Task Processing

```

Deferred Processing Using Binary Semaphore

ISR:

```

void EXTI0_IRQHandler(void)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    xSemaphoreGiveFromISR(
        xSem,
        &xHigherPriorityTaskWoken);

    portYIELD_FROM_ISR(
        xHigherPriorityTaskWoken);
}

```

Task:

```

void ProcessTask(void *arg)
{
    while(1)
    {
        xSemaphoreTake(

```

```

        xSem,
        portMAX_DELAY);

    ProcessEvent();
}
}

```

Deferred Processing Using Task Notification

ISR:

```

vTaskNotifyGiveFromISR(
    xTaskHandle,
    &xHigherPriorityTaskWoken);

```

Task:

```

ulTaskNotifyTake(
    pdTRUE,
    portMAX_DELAY);

```

Deferred Processing Using Queue

ISR:

```

xQueueSendFromISR(
    xQueue,
    &data,
    &xHigherPriorityTaskWoken);

```

Task:

```

xQueueReceive(
    xQueue,
    &data,
    portMAX_DELAY);

```

Advantages

- Fast interrupt handling

- Reduced interrupt latency
- Improved responsiveness
- Better CPU utilization
- Easier maintenance

Applications

- UART Reception
- ADC Completion
- DMA Completion
- Button Interrupt Processing
- Sensor Interrupt Handling

Gatekeeper Task

What is a Gatekeeper Task?

A Gatekeeper Task is a dedicated task responsible for managing access to a shared resource.

Other tasks do not access the resource directly.

Instead, they send requests to the Gatekeeper Task through a queue.

The Gatekeeper Task performs all resource operations.

Why Gatekeeper Task?

Suppose three tasks need UART access.

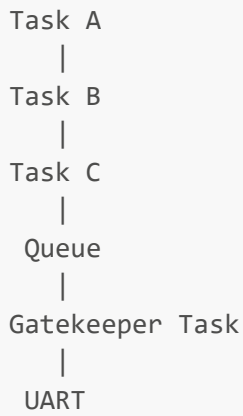
Without Gatekeeper:

```
Task A
Task B ---- UART
Task C
```

Problems:

- Mutex Required
- Race Conditions Possible
- Complex Synchronization

With Gatekeeper



Only Gatekeeper accesses UART.

No task directly touches UART.

Gatekeeper Using Queue

Create Queue

```

QueueHandle_t xUartQueue;

xUartQueue =
    xQueueCreate(
        10,
        sizeof(char *));
  
```

Sending Request

Producer Task:

```

char *msg = "Hello";

xQueueSend(
    xUartQueue,
    &msg,
    portMAX_DELAY);
  
```

Gatekeeper Task

```

void UartTask(void *arg)
{
    char *msg;
  
```

```

while(1)
{
    xQueueReceive(
        xUartQueue,
        &msg,
        portMAX_DELAY);

    printf("%s", msg);
}
}

```

Advantages of Gatekeeper Task

- No mutex required
- Eliminates race conditions
- Centralized resource control
- Easier debugging
- Scalable design

Typical Applications

UART Logging

```

Tasks
|
Queue
|
UART Task
|
UART

```

LCD Display Manager

```

Tasks
|
Queue
|
LCD Task
|
LCD

```

File System Access

```
Tasks
|
Queue
|
File Task
|
Storage Device
```

Interview Questions

What is Deferred Processing?

A technique where lengthy ISR work is postponed and executed by a task.

Why should ISRs be short?

To minimize interrupt latency and improve responsiveness.

What is a Gatekeeper Task?

A dedicated task that exclusively manages access to a shared resource.

Which FreeRTOS object is commonly used with Gatekeeper Tasks?

Queue.

What is the advantage of a Gatekeeper Task over Mutex?

A Gatekeeper Task completely avoids direct concurrent access to the resource and often eliminates the need for mutexes.